Fine-Tuning OpenVLA, and Fine-Tuning the Lens

A study guide to adapting OpenVLA (full fine-tuning, LoRA, the LIBERO recipe) and to *trained read-outs* (tuned lens, probes) for the perception→action question

Prepared for the VLAinterp project

October 2026

Abstract

"Fine-tuning" means two different things in this project, and mixing them up is the fastest way to get confused. *Fine-tuning the model* changes OpenVLA's weights so it behaves differently: this is how the base model became the LIBERO-Spatial checkpoint you study. *Fine-tuning a lens* trains a small read-out on top of a *frozen* OpenVLA, so you can measure what is already inside it. The model itself never changes. This guide covers both, in four parts: (I) what fine-tuning is mechanically, with a gradient-descent refresher; (II) how OpenVLA itself is fine-tuned: the strategies the authors compared, LoRA in detail, the released LIBERO configuration, and what that implies for interpretability; (III) lenses as trained read-outs: why the logit lens can mislead, how the tuned lens [1] fixes it, and how it relates to linear probes; (IV) a concrete plan for using a tuned lens to test the perception→action split suggested by your run 2 logit-lens results, including controls, the data you already have, and code sketches.

Contents

I What Fine-Tuning Is **3**

1 The one-paragraph mental model **3**

2 Gradient descent refresher **3**

II Fine-Tuning OpenVLA Itself **4**

3 The menu of strategies **4**

4 LoRA, in detail **5**

4.1 The idea 5

4.2 Merging 5

4.3 OpenVLA's released fine-tuning configuration 6

5 The LIBERO fine-tuning recipe (recap) **6**

6 What model fine-tuning means for interpretability **7**

III Lenses as Trained Read-Outs **7**

7 The logit lens, and why it can mislead **7**
7.1 Recap . . . 7
7.2 The hidden assumption . . . 7

8 The tuned lens **8**
8.1 Definition . . . 8
8.2 The four design choices that make it a lens, not a new model . . . 8
8.3 What the paper found . . . 9

9 The family of trained read-outs **9**

IV Applying It to the Perception→Action Question **10**

10 The hypothesis, stated precisely **10**

11 Predicted outcomes **11**

12 Specifics for OpenVLA **11**
12.1 Which vocabulary to normalize over . . . 11
12.2 Which positions, and one translator or several . . . 12
12.3 The data you already have, and its limit . . . 12

13 Controls that make the result convincing **12**

14 Implementation sketch **13**
14.1 Targets and inputs . . . 13
14.2 A low-rank translator per layer . . . 13
14.3 Training loop for one layer and one fold . . . 13

15 The two-minute spoken explanation **14**

16 Self-test questions **14**

17 Sources **15**

Part I
What Fine-Tuning Is

1 The one-paragraph mental model

Mental model: fine-tuning

Fine-tuning is just more training, starting from weights that already work. You keep the same machinery as the original training: a loss, gradients, an optimizer. You change three things: *where you start* (pretrained weights, not random ones), *what data you use* (a smaller, task-specific set), and *which weights are allowed to move* (all of them, some of them, or new small add-on weights).

The last choice is what separates the two meanings of fine-tuning in this project:

	Fine-tuning the model	Fine-tuning a lens
Goal	Change what the robot *does*	Measure what the model *already computes*
What moves	OpenVLA's own weights (or LoRA add-ons merged into them)	Only new read-out weights, e.g. one translator per layer
OpenVLA afterwards	A different model (new checkpoint)	Exactly the same model, bit for bit
Loss	Cross-entropy on demonstration actions	Usually KL to the model's *own* final prediction
Example	Base OpenVLA → `openvla-7b-finetuned-libero-spatial`	Logit lens → tuned lens

Common misconception: "fine-tune the logit lens" means "fine-tune W_U"

Not quite. W_U is part of OpenVLA. Retraining it would change the model you are trying to study. When people say "fine-tune the logit lens", they almost always mean training *extra* parameters that sit in front of a frozen W_U, one set per layer. This is the tuned lens (Part III). The original W_U is reused unchanged.

2 Gradient descent refresher

Every kind of training in this document, model or lens, runs the same loop:

1. **Forward pass.** Run a batch of examples through the network and compute the predictions.
2. **Loss.** One number measuring how wrong the predictions are: cross-entropy for OpenVLA, KL divergence for a tuned lens.
3. **Backward pass.** Compute the gradient $\nabla_\theta \mathcal{L}$: for every trainable number θ_i, which direction to move it to reduce the loss.

4. **Update.** Move each trainable number a small step against its gradient. The *optimizer* (SGD, Adam, AdamW) decides the exact step, and the *learning rate* sets its size.

Term	Meaning
Trainable / frozen	In PyTorch, `requires_grad=True` or `False`. Frozen weights still take part in the forward pass, but no gradient updates them.
Batch, step	A batch is the set of examples in one forward/backward pass. A step is one update.
Epoch	One full pass over the training data.
Learning rate (LR)	Step size. Too large and training diverges; too small and nothing changes.
AdamW	Adam (per-parameter adaptive step sizes with momentum) plus decoupled weight decay. The default for transformers.
Weight decay	A pull toward zero (or toward an initial value) on every update. It discourages large, overfit weights.
Held-out set	Examples never used for updates. They are the only honest measure of whether training generalizes.

Mental model: frozen vs. trainable

A frozen network is like a camera bolted to a tripod. You can still take pictures with it (forward passes), and you can learn to *interpret* its pictures better (train a lens), but nothing you do changes the camera. Fine-tuning the model means unbolting the camera and adjusting it.

Part II

Fine-Tuning OpenVLA Itself

3 The menu of strategies

OpenVLA's authors compared ways to adapt the pretrained model to a new robot setup (Franka tasks, batch size 16) [3]:

Strategy (what is trainable)	Success	Trainable params	VRAM
Full fine-tuning (everything)	69.7 ± 7.2%	7,188.1M	163.3 GB
Last layer only	30.3 ± 6.1%	465.1M	51.4 GB
Frozen vision (everything but the encoders)	47.0 ± 6.9%	6,760.4M	156.2 GB
Sandwich (vision + token embeddings + last layer)	62.1 ± 7.9%	914.2M	64.0 GB
LoRA, $r = 32$	**68.2 ± 7.5%**	**97.6M (1.4%)**	59.7 GB
LoRA, $r = 64$	68.2 ± 7.8%	195.2M	60.5 GB

How to read this.

- **Training only the last layer fails** (30%). Adapting to a new robot needs changes throughout the network, not just at the read-out.
- **Freezing the vision encoders hurts badly** (47%). The same lesson as in pretraining: robot control needs visual features the encoders don't produce out of the box.
- **LoRA matches full fine-tuning** while training 1.4% of the parameters, on one GPU in 10–15 hours. Going from rank 32 to 64 buys nothing. Every released LIBERO checkpoint uses LoRA $r = 32$.

4 LoRA, in detail

4.1 The idea

LoRA (Low-Rank Adaptation) [2] freezes every original weight matrix W and learns a *low-rank correction* next to it:

$$W_{\text{eff}} = W + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r \times d_{\text{in}}}, \;\; B \in \mathbb{R}^{d_{\text{out}} \times r}, \;\; r \ll d. \tag{1}$$

- A squeezes the input down to r numbers, and B expands them back. Their product BA has the same shape as W but rank at most r.
- B **starts at zero**, so $BA = 0$ and the model starts out exactly equal to the pretrained one. A starts random. (PEFT's default is Kaiming-uniform; OpenVLA sets `init_lora_weights="gaussian"`.)
- α/r is a fixed scale that keeps update sizes comparable across ranks.
- **Why it works:** the change needed to adapt a pretrained model tends to be low-dimensional. You don't need 16.8M free numbers to nudge a 4096×4096 matrix toward a new task.

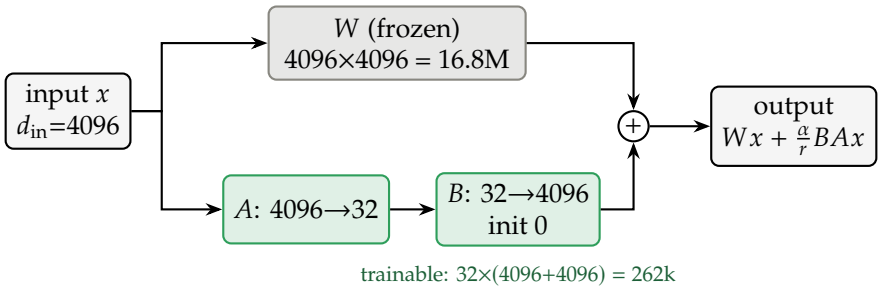


Figure 1: LoRA on one 4096×4096 projection. The frozen path and the low-rank path run in parallel and are added. Trainable parameters drop from 16.8M to 262k (64× fewer) for this one matrix.

4.2 Merging

After training, $\frac{\alpha}{r} BA$ can be added into W once (`merge_and_unload()` in PEFT). The result is an ordinary model with the original architecture and *zero* extra inference cost. The released LIBERO checkpoints are distributed this way, which is why you can load them exactly like the base model.

4.3 OpenVLA's released fine-tuning configuration

From `vla-scripts/finetune.py` in the OpenVLA repository:

Setting	Value
LoRA rank r	32
LoRA α	$\min(r, 16) = 16$, so scale $\alpha/r = 0.5$
LoRA dropout	0.0
Target modules	`"all-linear"`: every linear layer in the vision encoders, projector, and Llama
LoRA init	`"gaussian"`
Optimizer	AdamW, constant learning rate 5×10^{-4} (no schedule)
Batch size	16 (gradient accumulation 1)
Image augmentation	on (random crops etc., hence the center crop at test time)
Logged metrics	training loss, `action_accuracy` (exact bin match), `l1_loss` (on decoded continuous actions)

The loss is the same cross-entropy on action tokens as in pretraining. Only the data (LIBERO demonstrations) and the trainable parameters (LoRA matrices) change.

Interpretability note: was W_U touched?

PEFT's documentation says `"all-linear"` selects every linear layer *except the output layer* when the model is a Hugging Face `PreTrainedModel`. If that applies to OpenVLA's wrapper, the LIBERO checkpoint has **the same W_U as base OpenVLA**. Everything LIBERO-specific then lives in the low-rank changes to the layers that *produce* the residual stream. This is worth verifying directly rather than assuming: load both checkpoints and compare `lm_head.weight`. If they are identical, the LIBERO model learned to aim its stream at the *same* 256 action templates as the base model.

5 The LIBERO fine-tuning recipe (recap)

The architecture guide covers this in detail. In short, the data were prepared as follows before LoRA training:

1. Demonstrations re-rendered at 256×256 and rotated 180° (LIBERO renders upside down).
2. No-op actions filtered out (otherwise the policy learns to stand still), and failed demos dropped.
3. Third-person camera only. New per-suite action statistics (q01/q99), stored under a new `unnorm_key` such as `libero_spatial`.
4. One LoRA-r=32 run per suite, then merged.

6 What model fine-tuning means for interpretability

- **You are studying a fine-tuned model.** Your run 2 findings describe `openvla-7b-finetuned-libero-spatial`, not base OpenVLA. A natural control is to repeat a lens analysis on base OpenVLA (on Bridge-style inputs) to see which patterns come from pretraining and which from LIBERO fine-tuning.
- **LoRA changes are low-rank by construction.** The difference between the base and LIBERO weights in every layer has rank ≤ 32. That is a strong structural fact: whatever "LIBERO knowledge" was added lives in a small subspace of each layer. Comparing base and fine-tuned residual streams on the same inputs is a clean experiment.
- **Fine-tuning the model is not an interpretability tool.** It changes the object of study. For measuring, use frozen-model read-outs (Part III).

Part III
Lenses as Trained Read-Outs

7 The logit lens, and why it can mislead

7.1 Recap

At a read position (for Δx, the "_" token), the stream after layer ℓ is $x_\ell \in \mathbb{R}^{4096}$. The logit lens pretends the model stops there:

$$\text{LogitLens}_\ell(x_\ell) = W_U^{\text{act}} \, \text{RMSNorm}_{\text{final}}(x_\ell) \in \mathbb{R}^{256}, \tag{2}$$

where W_U^{act} is the 256 action rows of W_U. A softmax over those 256 scores gives the lens distribution q_ℓ. Comparing it to the final distribution p gives your run 2 metrics: agreement, $\pm k$ bins, KL, settle layer.

Common misconception: the Hugging Face hidden-state gotcha

In Hugging Face Llama, `hidden_states[0..31]` are raw residual streams, but `hidden_states[32]` has *already* been through the final RMSNorm. Your `lens_for_step` in `experiments.py` already handles this by applying the norm to every layer except the last. Any tuned-lens code must handle it the same way.

7.2 The hidden assumption

The logit lens assumes that layer ℓ writes information in *the same coordinate system* as layer 32, the only layer whose output was ever trained to be read by W_U. Nothing in training enforces that. Belrose et al. document two ways it breaks in language models [1]:

- **Representational drift.** The "basis" the residual stream uses changes with depth. Some dimensions carry huge variance in early layers and dominate the lens's output without meaning anything to W_U.

- **Bias.** Even when the right information is present, the logit lens is systematically skewed toward certain tokens. On some models (BLOOM, OPT-125M, GPT-Neo) it is badly unreliable. For example, it often just predicts the *input* token at the current position for half the layers.

Your run 2 result has a symptom of exactly this kind: early layers are *confident and wrong*, concentrating on a few default bins, such as bin 107 on 57% of steps at layer 8. That could mean early layers carry no action information. It could also mean they carry it in a basis W_U can't read.

Mental model: two stories the logit lens can't separate

(A) Real split. Layers 1–20 don't contain the action yet. They are doing perception, and the action is computed late.
(B) Translation failure. The action (or most of it) is already computed by mid-depth but encoded in a different basis. The late layers mostly *re-express* it in the format W_U reads.
Both predict the same logit-lens curves. You need a read-out that can translate.

8 The tuned lens

8.1 Definition

For each layer ℓ, learn an affine *translator* (A_ℓ, b_ℓ) and apply the ordinary logit lens to the translated stream:

$$\text{TunedLens}_\ell(x_\ell) = W_U^{\text{act}}\,\text{RMSNorm}_{\text{final}}\big(A_\ell\,x_\ell + b_\ell\big), \qquad A_\ell \in \mathbb{R}^{4096\times4096},\ b_\ell \in \mathbb{R}^{4096}. \tag{3}$$

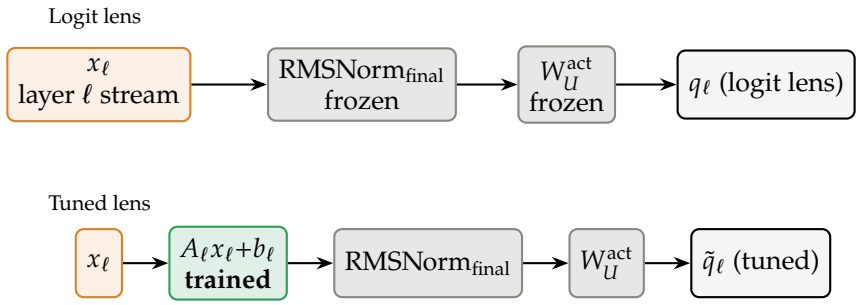


Figure 2: The tuned lens inserts one trained affine map per layer before the frozen read-out. Gray = frozen OpenVLA parts, green = the only trainable parameters.

8.2 The four design choices that make it a lens, not a new model

1. **Everything in OpenVLA is frozen**, including W_U and the final norm. Only A_ℓ, b_ℓ train.
2. **Initialize at the logit lens**: $A_\ell = I$, $b_\ell = 0$. Before training, the tuned lens *is* the logit lens. Training measures how far you have to move from it.
3. **Distill, don't supervise.** The target is the model's *own* final distribution p at the same position, not the ground-truth action:

$$\mathcal{L}_\ell = \mathbb{E}_{\text{inputs}}\Big[D_{\text{KL}}\big(p \,\|\, \tilde{q}_\ell\big)\Big] = \mathbb{E}\Big[\,\textstyle\sum_b p(b)\,\big(\log p(b) - \log \tilde{q}_\ell(b)\big)\Big]. \tag{4}$$

The lens is asked "can you recover what *this model* will say?", not "can you solve the robot task?". That keeps it from learning information the model doesn't use.

4. **Keep it affine.** A linear map plus bias can rotate, rescale, and shift a basis. It cannot do the multi-step nonlinear computation the remaining layers do. (A full 4096×4096 affine map is still quite powerful, which matters in Section 9.)

KL direction. $D_{\mathrm{KL}}(p \,\|\, \tilde{q})$ heavily penalizes the lens for putting low probability on bins the final layer likes. It is the same direction as the KL in your run 2 analysis, so the numbers are directly comparable.

Original training recipe. Belrose et al. trained translators with SGD + Nesterov momentum for 250 steps, learning rate 1.0 with linear decay, gradient clipping at 1, weight decay 10^{-3}, and about 2^{18} tokens per step. A later note in the paper reports that the Muon optimizer reached much lower KL and calls the original lenses "severely undertrained". The practical lesson: train until held-out KL stops improving, not for a fixed number of steps.

8.3 What the paper found

- Tuned-lens predictions are more accurate and much less biased than the logit lens's at every layer, on models up to 20B parameters.
- **Causal faithfulness:** directions that matter most to the tuned lens (found by ablation) are also the directions that matter most to the model's actual output (Spearman $\rho = 0.89$ in their Pythia-410M test). The lens reads features the model really uses.
- **Prediction depth** (the layer after which the lens's top-1 stops changing, i.e. your "settle layer") became more meaningful with the tuned lens. It correlates better with how hard an example is.

9 The family of trained read-outs

The tuned lens is one point on a spectrum. Moving down the table adds capacity: the read-out can find more, but it becomes harder to say whether it *found* information or *computed* it.

Read-out	Trained params / layer	Target	Question it answers
Logit lens	0	–	Is the answer already in W_U's format?
Bias-only lens	b_ℓ (4K)	final p	. . . after correcting a constant offset?
Low-rank tuned lens	$A_\ell = I + UV^\top$, rank k	final p	. . . after a small change of basis?
Tuned lens (full affine)	A_ℓ, b_ℓ (16.8M)	final p	. . . after any change of basis?
New unembedding per layer	256×4096 (1M)	final p	Is the final answer linearly decodable at all?
Linear probe (your ridge)	$4096 \times k$	ground truth	Is the true action / object position linearly decodable?
MLP probe	larger	ground truth	Is it decodable by *any* simple network?

Mental model: capacity vs. selectivity

A read-out with too little capacity misses information that is there: the logit lens's failure mode. A read-out with too much capacity "finds" information by computing it from raw ingredients: a deep probe can decode almost anything from almost any layer. Good practice is to report *several* rungs of the ladder and draw conclusions only from patterns that hold across them.

Tuned lens vs. your ridge probes. Your `-mode probe` regresses the *executed* action and object positions from each layer, with no W_U involved. The tuned lens asks whether layer ℓ can be translated into the model's *own* answer *through* W_U. They answer different questions, and agreement between them is strong evidence:

- Probe finds the action at layer 15, and so does the tuned lens: the action is computed by mid-depth (story B).
- Probe finds it at 15 but the tuned lens doesn't: the information exists but not in a form the model's read-out path uses yet.
- Neither finds it before layer 26: strong support for a real late computation (story A).

Part IV
Applying It to the Perception→Action Question

10 The hypothesis, stated precisely

From run 2 (logit lens, LIBERO-Spatial, 6,460 steps):

- Movement dimensions (x, y, z, roll, pitch, yaw) only beat the most-common-bin baseline at layers 25–28. Median settle layer is 28–31.
- The gripper settles much earlier (median layer 21), but mostly for "keep doing the same thing". Gripper *switches* form late.
- Early layers are confidently wrong, concentrated on a few default bins.

Hypothesis: layers $\lesssim 20$ do perception (scene and instruction), and the action is computed in the last ~8 layers. **What the tuned lens adds:** it separates "action computed late" (A) from "action computed earlier but expressed late" (B).

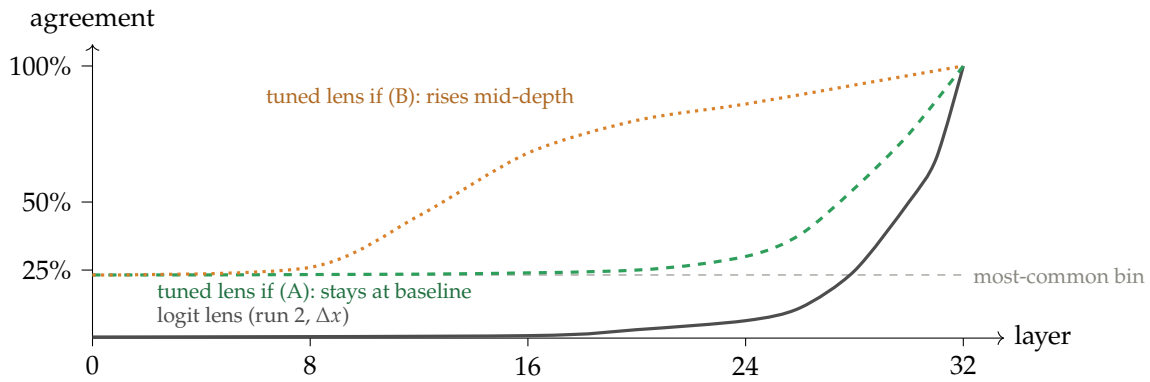


Figure 3: Schematic only, not data. The black curve is the run 2 logit-lens agreement for Δx. Note that *both* tuned-lens curves start at the most-common-bin baseline at layer 0: a trained bias alone can always output the most common bin. Beating the baseline is what counts, not beating the logit lens.

11 Predicted outcomes

Result	Interpretation
Tuned lens ≈ baseline until ~25, then rises	**(A) supported**, more strongly than before: even with a learned change of basis, the action isn't available mid-depth.
Tuned lens rises well above baseline by ~15–20	**(B)**: the action is mostly computed by mid-depth, and late layers re-express it for W_U. "Split" then describes representation format, not computation.
Low-rank lens ≈ full lens	The translation needed is simple (a few directions), and the conclusion doesn't depend on lens capacity.
Full lens ≫ low-rank lens at early layers	Be cautious: the full translator may be computing, not translating. Trust the low-rank result.
Gripper rises early, movement late	A per-dimension split, consistent with run 2's gripper finding.

12 Specifics for OpenVLA

12.1 Which vocabulary to normalize over

Two choices for the softmax:

- **256 action rows only** (recommended, and what run 2 does). The question is "which bin?". It matches all your existing metrics.
- **Full 32064 vocabulary.** It also tests whether early layers would rather output a text token. That's interesting, but it mixes two questions.

12.2 Which positions, and one translator or several

Each simulator step has 7 read positions: the "_" token for Δx, then each generated action token. The original tuned lens shares one translator per layer across all positions. For OpenVLA:

- **Start with Δx alone.** This is the data you already have (below).
- **Then all 7, with one shared translator per layer.** Seven times the training data per translator, and the most direct analogue of the paper.
- **Optionally one translator per (layer, dimension)**, if shared translators fit some dimensions much worse than others.

12.3 The data you already have, and its limit

`experiments.py` saves, for every step, the stream at *all 33 layers* for the *first* action position only (`rec["hidden"].append(hidden[0])`). So `runs/run2/hidden/*.npz` holds:

$$H \in \mathbb{R}^{6460\times 33\times 4096} \text{ (fp16, about 1.7 GB).}$$

Δx **is ready to go.** The other six dimensions need a new run that also saves `hidden[1:7]`, about 7× the storage (≈12 GB).

Common misconception: more parameters than examples

A full translator has $4096^2 \approx 16.8$M parameters. With 5-fold episode-held-out splits, about 5,200 training vectors per layer remain. The full affine lens can therefore *memorize* its training set. Its held-out score is the only number that means anything. Use some combination of:

- **Low rank:** $A_\ell = I + U_\ell V_\ell^\top$ with rank $k \in \{8, 32, 128\}$, V_ℓ initialized to zero so training starts at the logit lens. Report results across k.
- **Weight decay toward identity:** penalize $\|A_\ell - I\|^2$, not $\|A_\ell\|^2$.
- **Early stopping** on held-out-episode KL.
- **More data:** more rollouts, and using all 7 positions with a shared translator.

13 Controls that make the result convincing

1. **Hold out whole episodes.** Consecutive steps are near-duplicates. Reuse the `GroupKFold`-by-episode split from `run_probes`.
2. **Layer-0 control.** At the Δx read position, layer 0 is just the embedding of the "_" token, *identical at every step*. The tuned lens there can only output one fixed distribution, so its agreement must equal the most-common-bin rate (23.2% for x in run 2) and no more. A higher number means a bug or leakage.
3. **Compare to the baseline, not to the logit lens.** As Figure 3 shows, a trained bias alone beats the logit lens at early layers. The meaningful line is the most-common-bin baseline.
4. **Capacity sweep.** Bias-only, low-rank $k = 8/32/128$, full. Draw conclusions only from what is stable across them.

5. **Shuffled-input control.** Train on streams paired with targets from *other* steps. Held-out agreement should fall to baseline. If it doesn't, the lens is exploiting something other than step-specific information.
6. **Cross-check with probes.** Plot tuned-lens agreement next to your ridge-probe R^2 for the executed action at the same layers.
7. **Causal follow-up (later).** A read-out shows information is *present*. To show the model *uses* it, intervene: patch or ablate the directions the translator relies on at layer ℓ and check whether the final action changes. This is the step that turns a correlation into a mechanism.

14 Implementation sketch

14.1 Targets and inputs

```
# H: [N, 33, 4096] fp16 from runs/run2/hidden/*.npz (Delta-x position)
# groups: [N] episode index, for GroupKFold (load_hidden_dataset already does this)
norm, head = get_final_norm_and_head(model) # frozen, as in --mode lens
W_act = head.weight[action_ids].float() # [256, 4096], action rows of W_U

# Target: final distribution. hidden_states[32] is ALREADY normed in HF Llama.
with torch.no_grad():
    logp_final = torch.log_softmax(H[:, 32].float() @ W_act.T, dim=-1) # [N, 256]
```

14.2 A low-rank translator per layer

```
class Translator(nn.Module):
    def __init__(self, d=4096, rank=32):
        super().__init__()
        self.U = nn.Parameter(torch.randn(d, rank) / d**0.5)
        self.V = nn.Parameter(torch.zeros(rank, d)) # zero -> starts as the logit lens
        self.b = nn.Parameter(torch.zeros(d))
    def forward(self, x): # x: [B, d]
        return x + (x @ self.U) @ self.V + self.b # A = I + U V

def lens_logp(x, translator):
    z = norm(translator(x)) # frozen final RMSNorm
    return torch.log_softmax(z @ W_act.T, dim=-1) # [B, 256]

# KL(p_final || q_lens), averaged over the batch
def kl(logp_final, logq):
    return (logp_final.exp() * (logp_final - logq)).sum(-1).mean()
```

14.3 Training loop for one layer and one fold

```
for layer in range(32): # layer 32 needs no lens
    for train_idx, test_idx in GroupKFold(5).split(H, groups=groups):
        T = Translator(rank=k).cuda()
```

```
    opt = torch.optim.AdamW(T.parameters(), lr=1e-3, weight_decay=0.0)
    for epoch in range(max_epochs):
        for batch in minibatches(train_idx, 256):
            x = H[batch, layer].float().cuda()
            loss = kl(logp_final[batch], lens_logp(x, T))
            opt.zero_grad(); loss.backward(); opt.step()
        # early stopping on a validation split carved from train_idx
    # score on held-out episodes: agreement, +-k bins, KL, entropy
```

The OpenVLA weights are only needed to fetch W_U^{act} and the final norm, so training runs on cached vectors and is cheap: minutes per layer on one GPU. Output the same metrics and plots as `-mode lens` so the curves overlay directly.

15 The two-minute spoken explanation

> "The logit lens reads intermediate layers through OpenVLA's own output matrix. That assumes every layer stores information in the same format as the final layer, and nothing guarantees it. Our run 2 lens says movement actions only become readable around layer 26. That could mean the action is computed late, or that it's computed earlier but in a format the output matrix can't read. To separate those, I train a tuned lens: a small affine translator per layer, initialized to the identity, that maps that layer's residual stream into the final layer's format before the frozen output matrix reads it. The model stays completely frozen, and the translator is trained to match the model's own final prediction, not the true action, so it can't learn the task by itself. If the tuned lens still can't beat the most-common-bin baseline until the late layers, that's much stronger evidence that the action really is computed late. If it reads the action from mid-depth, the late layers are mostly translating. Controls: episode-held-out evaluation, a layer-0 sanity check, low-rank versions to limit capacity, and a comparison with linear probes on the true actions."

16 Self-test questions

1. What is the difference between fine-tuning OpenVLA and fine-tuning a lens? Which one changes the model?
2. In LoRA, why is B initialized to zero? What does merging do, and why does it make inference no slower?
3. How many trainable parameters does LoRA $r = 32$ add to one 4096×11008 matrix?
4. Why might the LIBERO checkpoint share its W_U with base OpenVLA, and how would you check?
5. State the two stories (A and B) the logit lens can't distinguish.
6. Why is the tuned lens trained against the model's final distribution rather than the ground-truth action?
7. Why does the tuned lens at layer 0 already score the most-common-bin rate, and why does that make "beats the logit lens" a weak claim?

8. With about 5,200 training vectors and 16.8M translator parameters, what goes wrong, and what are three remedies?
9. How would you tell whether a tuned lens is translating or computing?
10. What experiment would show the model *uses* the information a lens finds?

17 Sources

References

[1] N. Belrose, I. Ostrovsky, L. McKinney, Z. Furman, L. Smith, D. Halawi, S. Biderman, J. Steinhardt. *Eliciting Latent Predictions from Transformers with the Tuned Lens.* arXiv:2303.08112, 2023 (v6, 2025). https://arxiv.org/abs/2303.08112

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR 2022. https://arxiv.org/abs/2106.09685

[3] M. J. Kim, K. Pertsch, S. Karamcheti, et al. *OpenVLA: An Open-Source Vision-Language-Action Model.* CoRL 2024. https://arxiv.org/abs/2406.09246

[4] Hugging Face PEFT documentation, LoRA developer guide (`target_modules="all-linear"`, initialization, `merge_and_unload`). https://huggingface.co/docs/peft/developer_guides/lora

[5] OpenVLA fine-tuning script, `vla-scripts/finetune.py`. https://github.com/openvla/openvla

Run 2 numbers come from `docs/run2_logit_lens_analysis.md`. The implementation sketch is illustrative and has not been run. Learning rates and epoch counts are starting points to tune on held-out KL.